

Computational Lab 2

Hasan Latief

Email: hasan.latief@rutgers.edu

Differential Equations

April 9, 2025

Language Choice

For this assignment, I chose to use Haskell. My decision was partly motivated by the assignment's claim that I would need to use `for` loops. Since Haskell doesn't have them, I thought it would be an interesting challenge. Also, I thought it would be good opportunity to learn a functional programming language.

Exercises

Exercise 1 Write a routine, `integrateRiemann`, that takes a function F , two numbers $t_0, t \in \mathbb{R}$ and an integer $N \in \mathbb{N}$ and returns

$$\frac{t - t_0}{N} \sum_{n=0}^N F(t_0 + (t - t_0) \frac{n}{N}).$$

```
integrateRiemann :: (Double -> Double)
                  -> Double          -- t0
                  -> Double          -- t
                  -> Int             -- N
                  -> Double          -- Riemann Sum Estimate
integrateRiemann f t0 t n = dx * sum (map f xs)
  where
    dx = (t-t0) / (fromIntegral n)
    xs = [t0 + fromIntegral i * dx | i <- [0..n]]
```

Exercise 2 Write a routine `picard` that takes two functions v and x_j , two numbers $t_0, t \in \mathbb{R}$ and an integer $N \in \mathbb{N}$. The routine should return the Riemann sum approximation of

$$x_0 + \int_{t_0}^t v(x_j(s), s) ds.$$

```

-- Calculates 1 picard iteration
picard :: (Double -> Double -> Double) -- v(x,t)
        -> (Double -> Double)          -- xj
        -> Double                      -- x0
        -> Double                      -- t0
        -> Int                        -- Riemann steps (n)
        -> ( Double -> Double )        -- xj+1(t)
picard v xj x0 t0 n = \t -> x0 + integrateRiemann expr t0 t n
  where expr :: Double -> Double
        expr s = v (xj s) s

```

Strictly speaking, the preceding function outputs the function $x_{j+1}(t)$, not the value of $x_{j+1}(t)$. However, the purpose of this is to make iterating `picard` easier in the following exercise.

Exercise 3 Take $t_0 = 0$, $x_0 = 1$, $N = 100$, $v(x, t) = x \cos(t)$. Compute $x_1(1)$, $x_2(1)$, $x_3(1)$, and $x_4(1)$.

Here is a function to compute an arbitrary Picard iteration:

```

-- Calcualtes j picard iterations
picardIterate :: (Double -> Double -> Double) -- v(x,t)
               -> Double                      -- x0
               -> Double                      -- t0
               -> Int                        -- Riemann steps (n)
               -> Int                        -- Picard iterations (j)
               -> (Double -> Double)          -- xj(t)
picardIterate v x0 t0 n j
  | j == 0    = const x0                      -- x0(t) = x0
  | otherwise = picard v xPrev x0 t0 n
    where xPrev = picardIterate v x0 t0 n (j - 1)

```

Calling this function with the appropriate parameters gives the desired result.

```

ghci> print ((picardIterate ( \x t -> x * cos t ) 1 0 100 1 ) 1)
1.8491654840673433
ghci> print ((picardIterate ( \x t -> x * cos t ) 1 0 100 2 ) 1)
2.2088934108797633
ghci> print ((picardIterate ( \x t -> x * cos t ) 1 0 100 3 ) 1)
2.3109556520360153
ghci> print ((picardIterate ( \x t -> x * cos t ) 1 0 100 4 ) 1)
2.332767052266379

```

I also can confirm that calculating $x_5(1) \approx 2.3365$ took a long time. 4876 seconds to be exact.

Exercise 4 Write a routine `interpolate` whose arguments are a number t and two vectors $\mathbf{Y}$ and $\mathbf{T}$. The routine should return

$$Y_n + \frac{t - T_n}{T_{n+1} - T_n} (x_1(T_{n+1}) - x_1(T_n))$$

where n is the unique integer such that $t \in [T_n, T_{n+1})$.

First, let us concern ourselves with finding such n .

```
-- Returns the index where Tn<=t<Tn+1
findInterval :: [Double] -> Double -> Int -> Int -> Int
findInterval ts t low high
  | low > high = error "No interval found"
  | t < ts!!mid = findInterval ts t low (mid - 1)
  | t >= ts!!(mid+1) = findInterval ts t (mid + 1) high
  | otherwise = mid
where
  mid = (low + high) `div` 2
```

We use a binary search for performance reasons. Now the interpolation.

```
interpolate :: Double          -- t
             -> [Double]       -- Y
             -> [Double]       -- T
             -> Double          -- Interpolated Value
interpolate t ys ts
  | t == last ts = last ys
  | otherwise = ys!!n + (t-ts!!n)/(ts!!(n+1)-ts!!n)*(ys!!(n+1)-ys!!n)
  where n = findInterval ts t 0 ((length ts)-1)
```

The first guard is there because if $t = T_N$, then `findInterval` will cause a crash because T_N is not in any $[T_n, T_{n+1})$.

Exercise 5 Write a routine `integrateRiemannInterpolated` whose arguments are a function v , a vector $\mathbf{Y}$, two numbers $t_0, t \in \mathbb{R}$, an integer $N \in \mathbb{N}$, and a vector $\mathbf{T}$. The routine should return

$$\frac{t - t_0}{N} \sum_{n=0}^{N-1} v \left(x(t_0 + (t - t_0) \frac{n}{N}), t_0 + (t - t_0) \frac{n}{N} \right)$$

where $x(t)$ is interpolated from $\mathbf{Y}$.

I have noticed a pattern of needing evenly spaced points, so I also wrote `linspace`.

```

-- Makes n linearly spaced points between t0 and tF
linspace :: Double -- t0
         -> Double -- tF
         -> Int     -- N
         -> [Double] -- List of Tn's
linspace t0 tF n = [t0 + fromIntegral i * dt | i <- [0..n]]
  where dt = (tF - t0) / fromIntegral n

integrateRiemannInterpolated :: (Double -> Double -> Double) -- v(x,t)
                             -> [Double]                    -- Y
                             -> [Double]                    -- T
                             -> Double                      -- t0
                             -> Double                      -- t
                             -> Int                         -- N
                             -> Double                      -- Output
integrateRiemannInterpolated v ys ts t0 t n = dx * sum ( zipWith v (map x tList) tList)
  where x t = interpolate t ys ts
        dx = (t-t0) / fromIntegral n
        tList = init (linspace t0 t n)

```

Exercise 6 Write a routine `picardInterpolated` whose arguments are a function v , two vectors $\mathbf{Y}, \mathbf{T}$, three numbers $x_0, t_0 \in \mathbb{R}$, an integer $N \in \mathbb{N}$. The routine then returns a vector $\mathbf{X}$ with

$$X_i = x_0 + \int_{t_0}^{T_i} v(y(s), s) ds$$

where the integral is approximated using `integrateRiemannInterpolated`, and $y(s)$ is computed from $\mathbf{Y}$ using `interpolate`.

```

picardInterpolated :: (Double -> Double -> Double) -- v(x,t)
                  -> [Double]                    -- Y
                  -> [Double]                    -- T
                  -> Double                      -- x0
                  -> Double                      -- t0
                  -> Int                         -- N
                  -> Double                      -- X
picardInterpolated v ys ts x0 t0 n = [x0 + integrateRiemannInterpolated v ys ts t0 t n | t <- ts ]

```

Exercise 7.1 Use `picardInterpolated` to compute the first 10 Picard iterates for $x'(t) = \cos(t)x(t)$, $x(0) = 1$. Take $t_0 = 0$, $N = 100$, $t_F = 2\pi$. On the same graph, plot $x_0(t), x_5(t), x_{10}(t)$ and the exact solution $e^{\sin(t)}$.

Exercise 7.2 Repeat the previous computation with $N = 1000$.

0th, 5th, 10th Picard Iterations and Solution (N = 100)

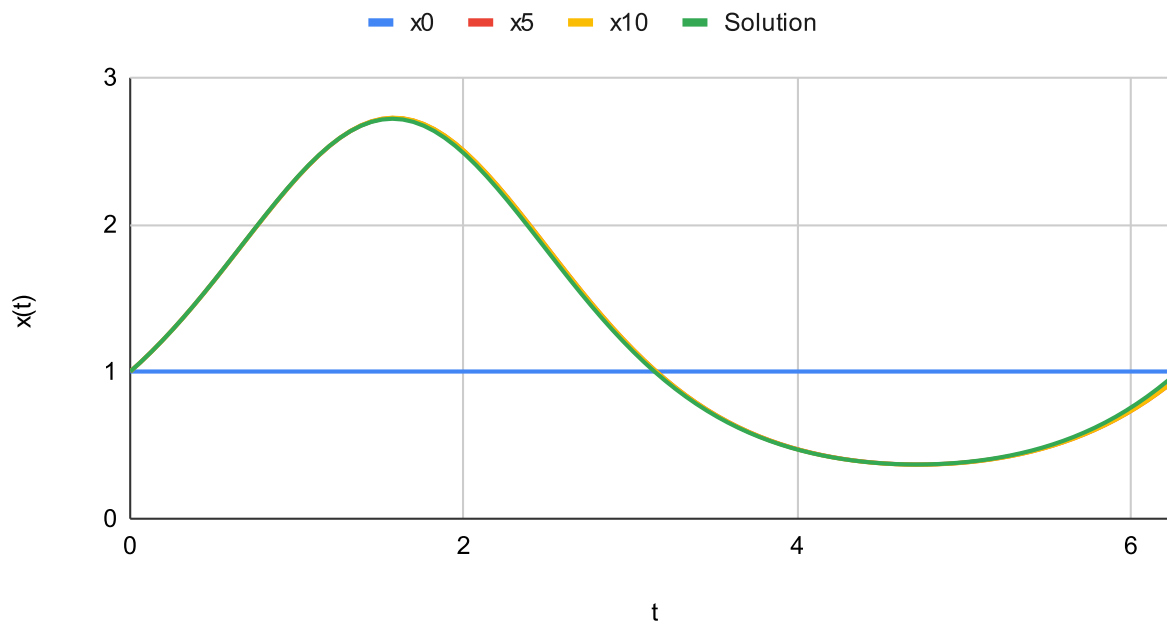


Figure 1: Approximation of $x(t)$, $N=100$

x_0 , x_5 , x_{10} and Solution (N = 1000)

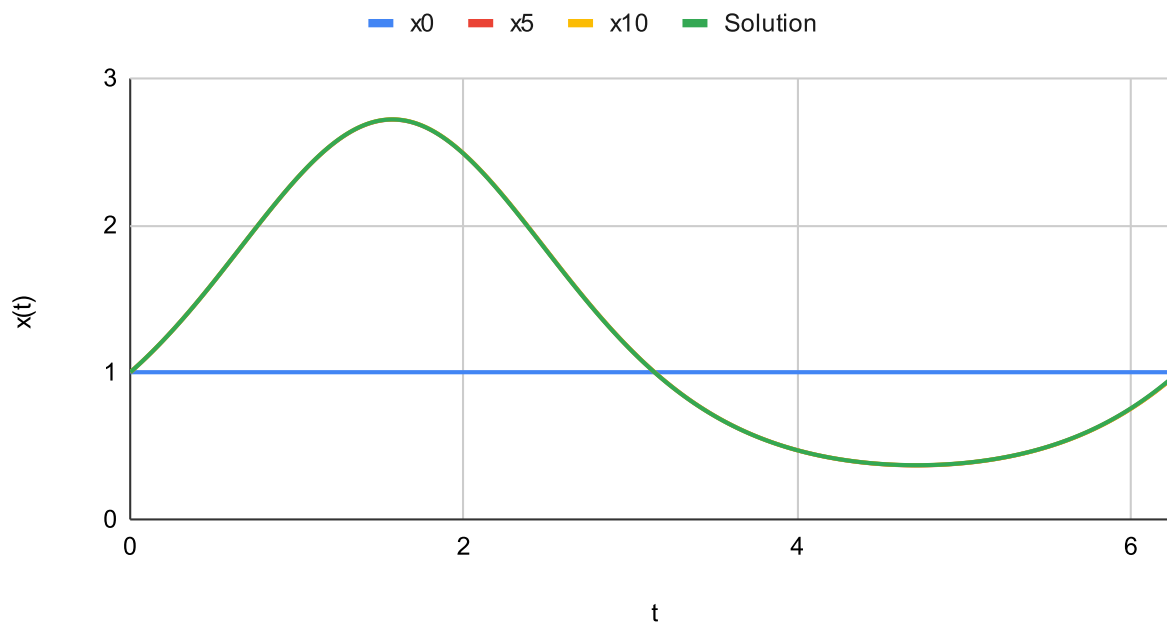


Figure 2: Approximation of $x(t)$, $N=1000$